

Unit_5(Automation and scripting for system administration and troubleshooting)

Scripting Languages (e.g., Bash and PowerShell)

A **scripting language** is a programming language used to automate repetitive tasks, manage systems, and execute commands without compiling the code. Scripts help system administrators perform tasks quickly and efficiently.

The two most common scripting languages for system administration are:

- **Bash** – Used mainly in Linux and Unix operating systems.
- **PowerShell** – Used mainly in Windows operating systems.

Bash (Bourne Again Shell)

Bash (Bourne Again Shell) is the default command-line shell and scripting language for most Linux and Unix systems. It is used to automate tasks such as file management, process control, and system administration.

Features of Bash

- Open-source and free
- Command-line interface (CLI)
- Supports shell scripting
- Automates repetitive tasks
- Strong file and process management
- Widely used in Linux servers

Common Bash Commands

Command	Function
ls	Lists files and directories
pwd	Displays the current directory
cd	Changes the current directory
mkdir	Creates a new directory
cp	Copies files or directories
mv	Moves or renames files
rm	Deletes files or directories
ps	Displays running processes

Example Bash Script

```
#!/bin/bash  
  
echo "Welcome to Linux"
```

```
date  
pwd
```

Output:

- Prints a welcome message.
- Displays the current date and time.
- Shows the current working directory.

PowerShell

PowerShell is a command-line shell and scripting language developed by Microsoft for automating system administration and configuration tasks in Windows.

Features of PowerShell

- Object-oriented scripting language
- Powerful automation capabilities
- Supports remote administration
- Built-in command library (cmdlets)
- Integrates with Windows Server and Active Directory
- Can manage both Windows and cross-platform systems

Common PowerShell Commands

Command	Function
Get-Process	Displays running processes
Get-Service	Displays services
Start-Service	Starts a service
Stop-Service	Stops a service
Restart-Service	Restarts a service
Get-ComputerInfo	Displays computer information
Get-Help	Shows help for commands
Get-ChildItem	Lists files and folders

Example PowerShell Script

```
Write-Host "Welcome to Windows Server"  
  
Get-Date  
  
Get-ComputerInfo
```

Output:

- Displays a welcome message.
- Shows the current date and time.
- Displays computer information.

Bash vs PowerShell

Feature	Bash	PowerShell
Operating System	Linux/Unix	Windows (also available on Linux and macOS)
Developer	GNU Project	Microsoft
Interface	Command-line shell	Command-line shell and scripting language
Data Handling	Text-based	Object-based
Primary Use	Linux system administration	Windows system administration
File Extension	.sh	.ps1
Package Management	Linux package managers	Windows administration tools

Uses of Scripting Languages

- Automating repetitive tasks
- User and group management
- File and directory management
- System monitoring
- Software installation
- Backup and recovery
- Network configuration
- Troubleshooting
- Scheduled maintenance

Advantages

- Saves time
- Reduces human errors
- Increases productivity
- Simplifies system administration
- Enables automation of complex tasks
- Supports remote management

Example

A company has **200 employees**.

Instead of manually creating 200 user accounts:

- A **PowerShell** script automatically creates all Windows user accounts.
- A **Bash** script automatically creates Linux user accounts, sets permissions, and configures home directories.

This saves significant time and ensures consistent configuration.

Automating Routine Tasks with Scripts

Automation is the process of performing repetitive or routine tasks automatically using scripts instead of doing them manually.

A **script** is a file containing a sequence of commands that a computer executes automatically. Scripts help system administrators save time, reduce errors, and improve efficiency.

Automating routine tasks with scripts means using scripting languages such as **Bash** (Linux) or **PowerShell** (Windows) to perform repetitive administrative tasks automatically without manual intervention.

Why Automate Routine Tasks?

- Saves time
- Reduces human errors
- Improves productivity
- Ensures consistency
- Simplifies system administration
- Allows administrators to manage many computers efficiently

Common Tasks That Can Be Automated

1. User Account Management

- Create user accounts
- Delete user accounts
- Reset passwords
- Add users to groups

2. File Management

- Copy files
- Move files
- Delete unnecessary files
- Create folders
- Rename files

3. Backup and Restore

- Schedule daily backups
- Compress backup files
- Restore important data

4. Software Installation

- Install applications automatically
- Update software
- Remove unwanted programs

5. System Monitoring

- Check CPU usage
- Monitor memory usage
- Check disk space
- Monitor network performance

6. Service Management

- Start services
- Stop services
- Restart services
- Check service status

7. Log Management

- Read system logs
- Delete old log files
- Generate reports

8. Scheduled Maintenance

- Clean temporary files
- Run updates
- Restart servers
- Perform health checks

Example Using Bash (Linux)

This script displays the current date and lists files in the current directory.

```
#!/bin/bash
```

```
echo "System Information"  
date  
ls
```

Explanation:

- `#!/bin/bash` → Specifies that the script uses the Bash shell.
- `echo` → Prints a message.
- `date` → Displays the current date and time.
- `ls` → Lists files and directories.

Example Using PowerShell (Windows)

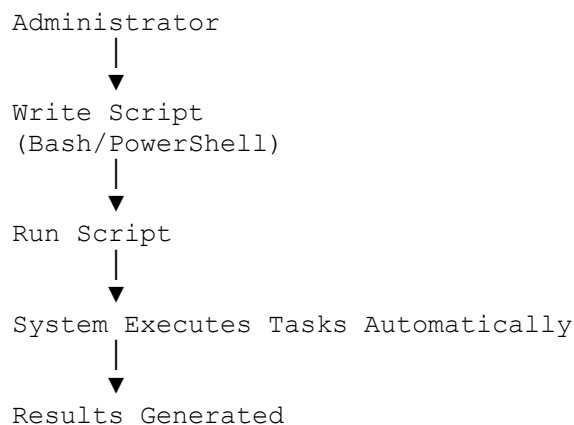
This script displays the current date and running services.

```
Write-Host "Windows Server Information"  
  
Get-Date  
  
Get-Service
```

Explanation:

- `Write-Host` → Displays text on the screen.
- `Get-Date` → Shows the current date and time.
- `Get-Service` → Lists all Windows services.

Automation Workflow



Advantages of Script Automation

- Faster execution of tasks
- Reduces repetitive manual work
- Minimizes mistakes
- Saves administrative costs

- Improves system reliability
- Enables scheduled and unattended execution
- Makes server management easier

Example

A company has **500 employee computers**.

Instead of manually updating antivirus software on every computer:

1. The administrator writes a **PowerShell script**.
2. The script automatically installs updates on all computers.
3. A scheduled task runs the script every night.
4. All systems stay updated without manual intervention.

Similarly, on a Linux server, a **Bash script** can automatically back up important files every day and remove old backup files.

Configuration Management Tools (e.g., Ansible)

In modern IT environments, managing the configuration of many servers manually is difficult and error-prone. **Configuration Management Tools** automate the installation, configuration, updating, and maintenance of servers, ensuring that all systems remain consistent.

One of the most popular configuration management tools is **Ansible**.

What is Configuration Management?

Configuration Management (CM) is the process of maintaining computers, servers, software, and network devices in a consistent, secure, and desired state throughout their lifecycle.

It ensures that all systems have the correct software, settings, and security configurations.

What is a Configuration Management Tool?

A **Configuration Management Tool** is software used to automate the configuration, deployment, and management of multiple systems from a central location.

Examples include:

- Ansible
- Puppet
- Chef
- Salt

Ansible

Ansible is an open-source configuration management, automation, and application deployment tool that automates IT tasks such as software installation, server configuration, and system management.

Unlike many other tools, Ansible is **agentless**, meaning it does not require special software (agents) to be installed on the managed machines. It usually communicates using **SSH** for Linux and **WinRM** for Windows.

Features of Ansible

- Open-source and free
- Agentless architecture
- Simple YAML-based playbooks
- Easy to learn and use
- Supports Linux and Windows
- Automates repetitive tasks
- Performs remote administration
- Scalable for managing hundreds or thousands of servers

Functions of Ansible

1. Server Configuration

Automatically configures operating systems and server settings.

2. Software Installation

Installs and updates software packages on multiple servers.

3. User Management

Creates, deletes, and manages user accounts and groups.

4. Application Deployment

Deploys applications consistently across multiple servers.

5. Security Configuration

Applies firewall rules, password policies, and security settings.

6. System Updates

Updates operating systems and installed software automatically.

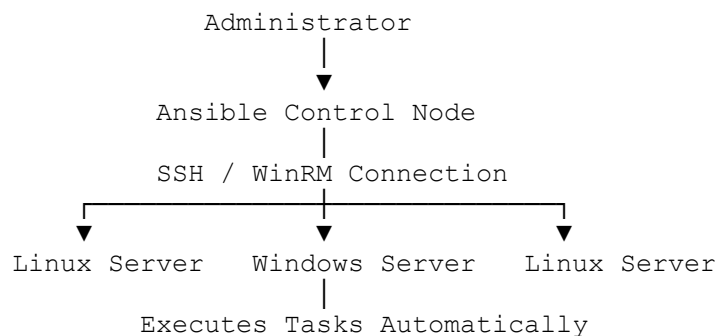
7. File Management

Copies, modifies, and deletes files on remote systems.

8. Service Management

Starts, stops, restarts, and checks system services.

How Ansible Works



Explanation:

1. The administrator writes an **Ansible Playbook**.
2. The playbook is executed from the **Ansible Control Node**.
3. Ansible connects to remote servers.
4. The required configuration changes are applied automatically.
5. All servers reach the desired configuration.

Advantages of Ansible

- Easy to learn (uses YAML syntax)
- No agents required
- Reduces manual work
- Ensures consistent configuration
- Saves time and cost
- Improves scalability
- Supports cloud and on-premises environments
- Simplifies automation

Limitations

- Can be slower than agent-based tools in very large environments.
- Requires SSH or WinRM connectivity.
- Complex workflows may require advanced playbook design.

Example

A company has **100 Windows servers** and **200 Linux servers**.

The administrator needs to:

- Install a web server.
- Update security patches.
- Create a new administrator account.
- Restart services.

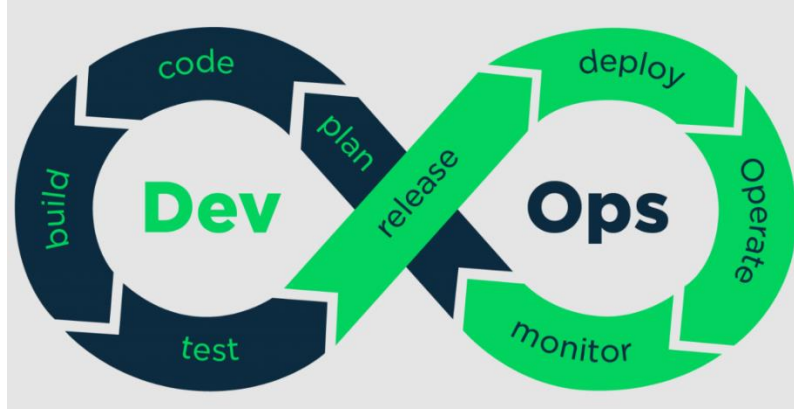
Instead of logging into each server manually, the administrator writes one **Ansible playbook**. Ansible connects to every server and performs the tasks automatically, ensuring all servers are configured consistently.

DevOps Principles and Practices

DevOps is a combination of **Development (Dev)** and **Operations (Ops)**. It is a software development approach that promotes collaboration, automation, continuous integration, continuous delivery, and continuous monitoring to deliver high-quality software quickly and reliably.

DevOps is a set of principles, practices, and tools that integrate software development and IT operations to improve collaboration, automate workflows, and accelerate software delivery while maintaining system stability.

DevOps Lifecycle



Objectives of DevOps

- Improve collaboration between development and operations teams.
- Automate software development and deployment.
- Deliver software faster.
- Improve software quality.
- Increase system reliability.
- Reduce deployment failures.

Principles of DevOps

1. Collaboration

- Developers and operations teams work together throughout the software lifecycle.
- Encourages communication and shared responsibility.

Benefit: Faster problem-solving and better teamwork.

2. Automation

- Automates repetitive tasks such as testing, deployment, configuration, and monitoring.
- Reduces manual work and human errors.

Example: Automatic software deployment using scripts.

3. Continuous Integration (CI)

- Developers frequently merge code into a shared repository.
- Every code change is automatically built and tested.

Benefits:

- Detects bugs early.

- Improves code quality.

4. Continuous Delivery (CD)

- Software is automatically prepared for release after successful testing.
- Ensures software is always ready for deployment.

Benefit: Faster and more reliable releases.

5. Continuous Monitoring

- Monitors application performance, servers, and infrastructure.
- Detects issues before they affect users.

Benefit: Improves reliability and availability.

6. Continuous Feedback

- Collects feedback from users, testing, and monitoring.
- Helps developers improve future releases.

7. Infrastructure as Code (IaC)

- Infrastructure is managed using code instead of manual configuration.
- Ensures consistent and repeatable deployments.

Example: Using configuration management tools such as Ansible.

8. Security (DevSecOps)

- Security is integrated throughout the development lifecycle.
- Automated security testing and vulnerability scanning are performed.

DevOps Practices

1. Version Control

- Stores and tracks source code changes.
- Enables multiple developers to collaborate.

Example: Git.

2. Continuous Integration (CI)

- Automatically builds and tests code whenever changes are committed.

3. Continuous Delivery/Deployment (CD)

- Automatically deploys tested applications to production or staging environments.

4. Automated Testing

- Runs unit, integration, and system tests automatically.

5. Configuration Management

- Maintains consistent server configurations.

Example: Ansible.

6. Monitoring and Logging

- Continuously monitors applications and infrastructure.
- Collects logs to troubleshoot issues.

7. Containerization

- Packages applications with all dependencies for consistent execution.

Example: Docker.

8. Orchestration

- Automates deployment and management of containers.

Example: Kubernetes.

Benefits of DevOps

- Faster software delivery

- Better collaboration
 - Higher software quality
 - Reduced deployment failures
 - Faster bug detection and resolution
 - Improved customer satisfaction
 - Increased automation
 - Better system reliability
 - Continuous improvement
-

Challenges of DevOps

- Cultural changes within organizations
- Learning new tools and technologies
- Initial setup costs
- Security management
- Complex automation pipelines

Example

A software company develops a banking application.

1. Developers write new code.
2. The code is stored in **Git**.
3. A **CI** pipeline automatically builds and tests the application.
4. If tests pass, the application is deployed using a **CD** pipeline.
5. **Ansible** configures the servers.
6. **Docker** packages the application.
7. **Kubernetes** manages the containers.
8. Monitoring tools continuously check application performance and generate alerts if problems occur.

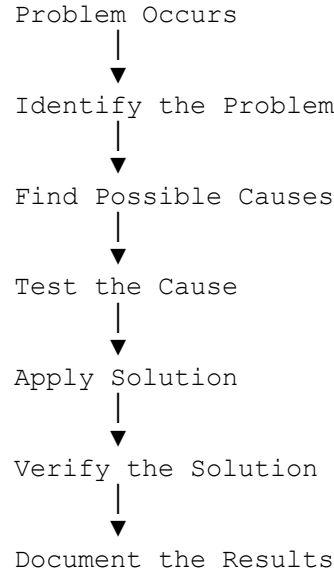
This workflow enables frequent, reliable, and automated software releases

Troubleshooting Methodology

Troubleshooting Methodology is a **systematic process** used to identify, diagnose, and resolve hardware, software, network, or operating system problems. It helps administrators solve issues efficiently while minimizing downtime.

Troubleshooting Methodology is a step-by-step approach used to find the root cause of a problem, apply an appropriate solution, verify that the issue is resolved, and document the process.

Troubleshooting Flow



Steps in Troubleshooting Methodology

1. Identify the Problem

- Gather information from the user.
- Observe symptoms and error messages.
- Determine what changed before the problem occurred.

Example Questions:

- What is the problem?
- When did it start?
- Has any new software or hardware been installed?

2. Establish a Theory of the Cause

- Analyze the collected information.
- Identify possible causes of the problem.
- Start with the simplest and most likely cause.

Example:

If a computer cannot access the internet, possible causes are:

- Loose network cable
- Incorrect IP address
- DNS issue
- Router failure

3. Test the Theory

- Perform tests to confirm or reject the suspected cause.
- If the theory is incorrect, develop a new theory.

Example:

- Check the network cable.
- Ping the gateway.
- Verify IP configuration.

4. Establish a Plan of Action

- Decide how to fix the problem.
- Consider the impact on users and the system.
- Back up important data if necessary.

5. Implement the Solution

- Apply the chosen solution.
- Follow organizational procedures.
- Monitor the system during implementation.

Example:

- Restart a service.
- Update drivers.
- Replace faulty hardware.
- Reconfigure network settings.

6. Verify Full System Functionality

- Ensure the problem is completely resolved.
- Test all related functions.
- Confirm that no new issues have been introduced.

7. Document the Problem and Solution

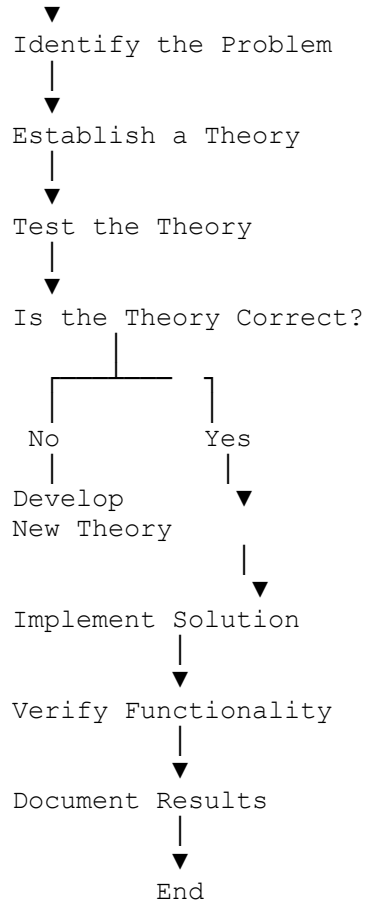
- Record:
 - Problem description
 - Root cause
 - Solution applied
 - Test results
 - Recommendations

Documentation helps solve similar problems more quickly in the future.

Troubleshooting Flowchart

Start

|



Example

Problem:

A user cannot print documents.

Step 1: Identify

- Printer shows "Offline."

Step 2: Theory

Possible causes:

- Printer is powered off.
- Network cable is disconnected.
- Print Spooler service has stopped.

Step 3: Test

- Check printer power.
- Verify network connection.
- Restart the Print Spooler service.

Step 4: Plan

Restart the Print Spooler service.

Step 5: Implement

Restart the service.

Step 6: Verify

Print a test page successfully.

Step 7: Document

Record that the Print Spooler service had stopped and restarting it resolved the issue.

Advantages of a Troubleshooting Methodology

- Provides a structured approach.
- Saves time.
- Reduces unnecessary changes.
- Identifies the root cause.
- Improves system reliability.
- Helps prevent recurring problems.
- Supports better documentation and knowledge sharing.

Example

A company employee cannot log in to the Windows domain.

1. The administrator checks the username and password.
2. Verifies network connectivity.
3. Confirms the **Domain Controller** is available.
4. Checks whether the user account is locked.
5. Unlocks the account and resets the password if necessary.
6. Tests the login again.
7. Documents the issue and the solution.

Case Studies in Troubleshooting

A **case study in troubleshooting** is a real-world example that demonstrates how a systematic troubleshooting methodology is used to identify, diagnose, and resolve computer, server, or network problems.

Case Study 1: User Cannot Log In to Windows Server

Problem

A user cannot log in to the Windows domain.

Troubleshooting Steps

Step 1: Identify the Problem

- User receives "Invalid username or password."

Step 2: Establish a Theory

Possible causes:

- Incorrect password
- Account locked
- Domain Controller unavailable

Step 3: Test the Theory

- Check username and password.
- Verify network connectivity.
- Check if the account is locked.

Step 4: Implement the Solution

- Unlock the account.
- Reset the password if required.

Step 5: Verify

- User logs in successfully.

Step 6: Document

- Record the issue and solution for future reference.

Case Study 2: Computer Has No Internet Connection

Problem

A user cannot access the internet.

Troubleshooting Steps

Step1.Identify

- Browser shows "No Internet."

Step2:Possible Causes

- Loose network cable

- Incorrect IP configuration
- DNS problem
- Router failure

Step3:Test

- Check network cable.
- Run `ipconfig`.
- Ping the default gateway.
- Test DNS resolution.

Step4:Solution

- Correct the IP settings or reconnect the cable.

Step5:Verification

- Open a website successfully.

Step 6: Document the Problem and Solution

Record the details for future reference.

Item	Details
Problem	Computer could not access the internet
Symptoms	Browser displayed "No Internet"
Root Cause	Incorrect IP configuration (or loose network cable)
Solution	Corrected IP settings and reconnected the cable
Verification	Successfully opened websites and confirmed internet connectivity
Recommendation	Check network cables regularly and use automatic IP configuration (DHCP) where appropriate

Case Study 3: Windows Server Running Slowly

Problem

A Windows Server responds very slowly.

Troubleshooting Steps

Identify

- High CPU and memory usage.

Possible Causes

- Too many running applications
- Malware infection
- Low disk space

Test

- Open Task Manager.
- Check CPU, RAM, and disk usage.
- Scan for malware.

Solution

- Stop unnecessary processes.
- Free disk space.
- Remove malware.
- Restart the server if needed.

Verification

- Server performance returns to normal.

6. Documentation

Record the following information:

- **Problem:** Web service failed to start.
- **Root Cause:** Incorrect service configuration (or the actual cause found).
- **Tests Performed:** Checked Event Viewer, dependencies, and permissions.
- **Solution Applied:** Corrected configuration, started dependent services, and restarted the service.
- **Result:** Service started successfully, and the website became accessible.
- **Date and Technician:** Record when the issue was resolved and who resolved it.

Best Practices in Troubleshooting

- Gather complete information before making changes.
- Start with the simplest and most likely causes.
- Make one change at a time.
- Back up important data before major changes.
- Test the solution thoroughly.
- Document the problem and the resolution.
- Learn from previous troubleshooting cases.

Advantages of Case Studies

- Improve practical problem-solving skills.
- Help understand real-world IT issues.
- Build confidence in diagnosing problems.
- Provide reusable solutions for similar incidents.
- Reduce troubleshooting time in the future.

Remote Administration

Remote Administration is the process of managing, monitoring, and troubleshooting a computer or server from another location over a network. It enables system administrators to perform administrative tasks without being physically present at the server.

Remote administration is widely used in businesses, educational institutions, cloud environments, and data centers to manage Windows Server and Linux systems.

Remote Administration is the ability to access and control a remote computer or server through a network using specialized tools and protocols.

Objectives of Remote Administration

- Manage servers from any location.
- Reduce travel time and maintenance costs.
- Troubleshoot systems remotely.
- Install software and updates.
- Monitor server performance.
- Improve productivity and efficiency.

Common Remote Administration Tools

Tool	Purpose
Remote Desktop (RDP)	Provides a graphical remote desktop for Windows servers and PCs.
Windows PowerShell Remoting	Executes PowerShell commands on remote Windows computers.
SSH (Secure Shell)	Secure command-line access to Linux/Unix systems and supported Windows systems.
Remote Server Administration Tools (RSAT)	Allows administrators to manage Windows Server roles and features from another Windows computer.

Tool	Purpose
Virtual Network Computing (VNC)	Provides remote graphical desktop access across different operating systems.

Functions of Remote Administration

1. Remote Server Management

- Configure server settings.
- Manage users and groups.
- Install roles and features.

2. Software Installation

- Install and update software remotely.
- Deploy applications to multiple computers.

3. Troubleshooting

- Diagnose system problems.
- Restart services.
- Check event logs.
- Resolve configuration issues.

4. System Monitoring

- Monitor CPU, memory, disk, and network usage.
- Check server health and performance.

5. User Management

- Create and delete user accounts.
- Reset passwords.
- Assign permissions.

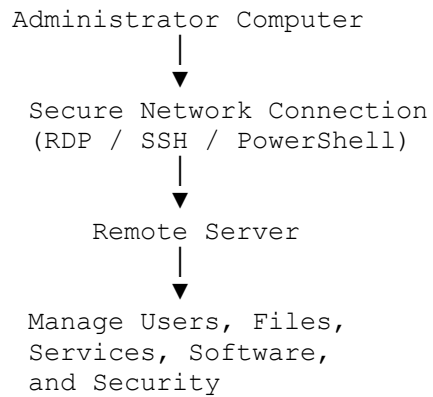
6. File Management

- Copy, move, delete, and modify files on remote systems.

7. Security Management

- Configure firewalls.
- Apply security updates.
- Monitor login activities.

Remote Administration Process



Advantages

- Access servers from anywhere.
- Saves time and travel costs.
- Faster troubleshooting and maintenance.
- Centralized administration.
- Improved productivity.
- Supports remote work.
- Minimizes system downtime.

Limitations

- Requires a stable network connection.
- Unauthorized access is possible if security is weak.
- Remote sessions may be slower over low-bandwidth networks.
- Proper authentication and permissions are required.

Security Best Practices

- Use **strong passwords** and **Multi-Factor Authentication (MFA)**.
- Enable **encrypted connections** (such as RDP with Network Level Authentication or SSH).
- Restrict remote access to authorized administrators.
- Keep systems updated with security patches.
- Monitor and audit remote login activities.
- Use firewalls to limit access to remote management services.

Example

A company has a **Windows Server** located at its head office. The system administrator is working from home.

Using **Remote Desktop (RDP)** or **PowerShell Remoting**, the administrator:

- Logs into the server remotely.
- Creates a new employee account.
- Installs security updates.
- Restarts a service that has stopped.
- Reviews event logs to troubleshoot an issue.

All of these tasks are completed without physically visiting the office.